

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 4

Duration: 1 Hour

Total: Marks:40

Annexure A

5

Code of Conduct:

I A Sai Swetha(192319072) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

INDUSTRY PROBLEM TASK

INDUSTRY SCENARIO

Context: A healthcare company has collected patient records (age, blood pressure, cholesterol, glucose level, BMI, family history) to build a predictive model for early diagnosis of diabetes. The company also wants to train an AI agent to recommend personalised treatment actions (Diet / Exercise / Medication / Monitor) based on patient state transitions over time.

You are hired as an AI/ML Engineer. Address the following tasks:

Task 1: Data Preparation & Inductive Learning

(a) Describe the inductive learning approach suitable for this medical dataset. Identify the target variable and at least three key features with justification.

(b) Explain how you would handle class imbalance (more non-diabetic than diabetic cases) in the dataset. Suggest and justify a suitable balancing strategy (SMOTE / under-sampling / class weights).

(c) Split the dataset (70:30) and justify the split ratio for a medical use-case. State the preprocessing steps (normalisation, encoding) applied and their impact.

1(a) Inductive Learning Approach

Definition

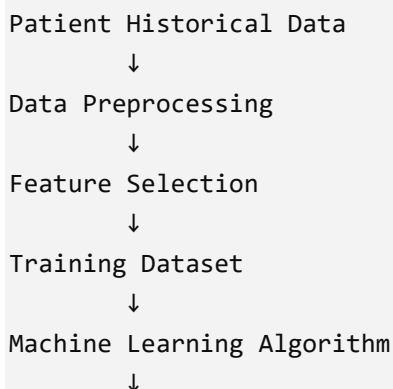
Inductive learning is a machine learning approach in which a model learns general patterns or rules from a set of labelled examples and uses those learned patterns to make predictions on new, unseen data.

For this healthcare problem, the patient dataset contains several input features and a target variable indicating whether the patient is diabetic or non-diabetic.

Suitable approach

A suitable approach is **supervised inductive learning**.

The process is:



Learned Diabetes Prediction Model



New Patient Data



Diabetes / Non-Diabetes Prediction

The model learns from previous patient records where the diabetes status is already known.

Target Variable

The target variable should be:

Diabetes Status

It can be represented as:

0 → Non-diabetic

1 → Diabetic

Therefore, this is a **binary classification problem**.

Important Features

At least three important features can be selected.

1. Glucose Level

Glucose level is one of the most important predictors because abnormal blood glucose is directly associated with diabetes.

A higher glucose level generally increases the likelihood of diabetes.

Therefore:

Glucose → Strong predictor

2. BMI

BMI represents the relationship between body weight and height.

Higher BMI can be associated with increased risk of insulin resistance and type 2 diabetes.

Therefore:

BMI → Important metabolic risk factor

3. Blood Pressure

Blood pressure provides information about cardiovascular and metabolic health.

Abnormal blood pressure can occur along with metabolic disorders and can contribute to overall diabetes risk assessment.

Therefore:

Blood Pressure → Useful clinical predictor

Other useful features

Age

Diabetes risk can increase with age, although diabetes can occur at younger ages as well.

Cholesterol

Abnormal cholesterol levels can indicate metabolic and cardiovascular risk.

Family History

A family history of diabetes can indicate increased genetic susceptibility.

Feature Summary

Feature	Importance
Glucose	Very high
BMI	High
Blood pressure	High
Age	Moderate/High
Cholesterol	Moderate

Feature	Importance
Family history	Important categorical risk factor

The assessment specifically identifies age, blood pressure, cholesterol, glucose, BMI and family history as available patient features.

1(b) Handling Class Imbalance

The dataset contains more non-diabetic patients than diabetic patients.

For example:

```
Non-diabetic = 900
Diabetic = 100
```

This produces a highly imbalanced dataset.

If we train a model directly, it may become biased toward the majority class.

For example, a model could predict almost every patient as non-diabetic and still obtain:

Although the accuracy is 90%, the model is clinically poor because it fails to identify diabetic patients.

Suitable Strategy: SMOTE

For this problem, **SMOTE (Synthetic Minority Over-sampling Technique)** is a suitable approach.

SMOTE generates synthetic examples of the minority class rather than simply duplicating existing observations.

Working

Suppose the diabetic class is the minority class.

SMOTE:

1. Selects a diabetic sample.
2. Finds neighbouring diabetic samples.
3. Creates a synthetic point between them.
4. Repeats the process until the minority class is sufficiently represented.

Conceptually:

```
Existing diabetic patients
      ↓
Find similar diabetic neighbours
      ↓
Generate synthetic diabetic samples
      ↓
Balanced training dataset
```

Why SMOTE is suitable

SMOTE is useful because:

- It increases minority-class representation.
- It reduces bias toward non-diabetic patients.
- It does not simply duplicate the same patients.
- It can improve recall for the diabetic class.

Important precaution

SMOTE should be applied **only to the training data**, not before splitting the entire dataset.

Correct:

```
Original Dataset
      ↓
Train/Test Split
      ↓
SMOTE on Training Data only
      ↓
Model Training
      ↓
Testing on untouched Test Data
```

If SMOTE is applied before splitting, synthetic information can leak into the test set and produce overly optimistic performance.

Alternative: Class Weights

Another suitable approach is:

Python

Run

```
DecisionTreeClassifier(  
    class_weight="balanced"  
)
```

Class weighting increases the penalty for misclassifying minority-class patients.

For a medical application, **SMOTE combined with appropriate class weighting or threshold adjustment** can be considered, but the final strategy should be validated carefully because synthetic observations may not perfectly represent real patients.

1(c) 70:30 Train-Test Split

The assessment specifies a **70:30 split**.

CO4-AT2

Therefore:

- **70% → Training data**
- **30% → Testing data**

Example

For 1,000 patients:

Training samples:

700

Testing samples:

Testing samples:

300

Why 70:30?

A 70:30 split provides:

- Enough data for the model to learn patterns.

- A sufficiently large independent test set.
- More reliable performance evaluation.
- Better assessment of generalisation to unseen patients.

This is particularly important in healthcare because a model should not only perform well on the data used to train it.

Preprocessing

1. Handling Missing Values

Missing values can be handled using:

- Mean/median imputation for numerical variables.
- Mode imputation for categorical variables.

For example:

Missing glucose → Median glucose

Missing BMI → Median BMI

Missing family history → Most frequent category

Median is often preferred for numerical medical variables when outliers are present.

2. Encoding Categorical Variables

Family history may be represented as:

Yes → 1

No → 0

For multiple categories, one-hot encoding can be used.

3. Normalisation

Features such as age, glucose, BMI and blood pressure can have different numerical ranges.

Standardisation can be performed using:

where:

- = original value
- = mean
- = standard deviation

Impact

Normalisation:

- Places numerical features on comparable scales.
- Is especially useful for Logistic Regression and other distance/gradient-based models.
- Usually has little direct effect on the basic splitting mechanism of a Decision Tree.

Task 2: Decision Tree for Diagnosis

(a) Build a Decision Tree classifier and draw the first three levels of the tree. Justify the root node selection using Information Gain / Gini Index values.

(b) Evaluate the model: report Accuracy, Precision, Recall, and F1-Score. Identify False Negatives (missed diabetes cases) and suggest how to reduce them—justify clinically.

(c) Apply post-pruning (cost-complexity pruning). Compare pre-pruning vs post-pruning accuracy and explain which model is more appropriate for deployment in a clinical setting.

2(a) Building the Decision Tree

A Decision Tree can be used to predict:

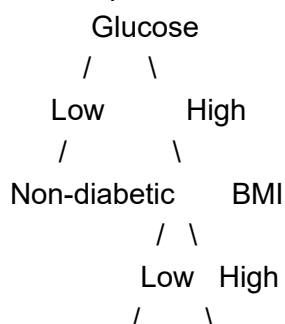
Patient

↓

Diabetic / Non-diabetic

The tree repeatedly divides patients according to feature values.

For example:



Monitor Diabetic

The actual tree depends on the dataset.

Gini Index

The Gini impurity is:

For binary classification:

A pure node has:

A node containing an equal mixture of two classes has maximum impurity:

Information Gain

Information Gain is based on entropy.

Information Gain is:

The feature that produces the largest reduction in impurity is selected.

Root Node

For this healthcare dataset, **glucose level would be a strong candidate for the root node** because it is closely associated with diabetes status.

However, the exact root node **cannot be determined from the assessment document alone**, because the actual patient dataset and its numerical values are not provided.

Therefore, the correct procedure is to calculate Gini/Information Gain for every candidate feature and select the one with the best split.

Example

Suppose the calculated results were:

Feature	Gini after split
Glucose	0.24
BMI	0.32
Blood Pressure	0.36
Age	0.39
Cholesterol	0.40

The smallest weighted Gini value would be for glucose.

Therefore:

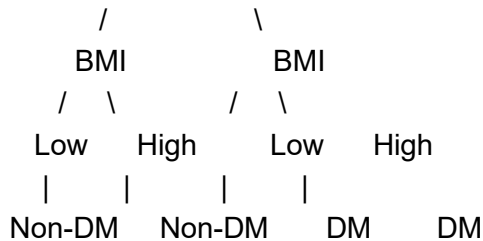
Root node = Glucose

because it produces the greatest reduction in impurity.

First Three Levels

A possible structure is:

```
      Glucose
     /   \
    /     \
  Low     High
```



This is an **illustrative structure**. The exact first three levels must be generated from the actual dataset.

Python Implementation

Python

Run

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    stratify=y,
    random_state=42
)
```

```
model = DecisionTreeClassifier(
    criterion="gini",
    max_depth=3,
    class_weight="balanced",
    random_state=42
)
```

```
model.fit(X_train, y_train)
```

```
plot_tree(
    model,
    feature_names=X.columns,
    class_names=["Non-diabetic", "Diabetic"],
    filled=True
)
```

2(b) Evaluation Metrics

The four required metrics are:

- Accuracy
- Precision
- Recall
- F1-score

CO4-AT2

Accuracy

Accuracy represents the overall percentage of correct predictions.

Precision

Precision answers:

Of all patients predicted as diabetic, how many were actually diabetic?

Recall

Recall is extremely important in diabetes screening.

It answers:

Of all actual diabetic patients, how many did the model identify?

F1-Score

F1-score provides a balance between precision and recall.

False Negatives

A **False Negative (FN)** occurs when:

Actual = Diabetic

Predicted = Non-diabetic

Example:

Actual	Predicted	Classification
Diabetic	Non-diabetic	False Negative

Why are false negatives important?

In a medical screening system, missing a diabetic patient can delay:

- Further diagnostic testing
- Clinical assessment
- Lifestyle intervention
- Monitoring
- Appropriate treatment

Therefore, the system should give particular importance to **recall/sensitivity**.

Reducing False Negatives

Several methods can be used:

1. Class weighting

Give a higher penalty to diabetic-class errors.

Python

Run

```
class_weight="balanced"
```

2. SMOTE

Increase representation of the minority diabetic class in training data.

3. Decision threshold adjustment

For probabilistic models, the threshold can be lowered so that more high-risk patients are flagged.

4. Improve feature quality

Use reliable measurements such as:

- Glucose
- BMI
- Blood pressure
- Age
- Family history

Clinical justification

For a screening system, **missing a diabetic patient can be more harmful than sending a patient for additional evaluation.**

Therefore, the model should prioritise sensitivity while maintaining acceptable specificity.

However, the AI system should be considered a **decision-support tool**, not a replacement for clinical diagnosis.

2(c) Post-Pruning

A Decision Tree can become too complex and overfit the training data.

Pre-Pruning

Pre-pruning stops the tree from growing too much.

Parameters include:

Python

Run

max_depth

min_samples_split

min_samples_leaf

Example:

Python

Run

```
DecisionTreeClassifier(  
    max_depth=4,  
    min_samples_leaf=10  
)
```

Post-Pruning

Post-pruning first allows the tree to grow and then removes branches that contribute little predictive value.

The assessment specifically asks for **cost-complexity pruning**.

CO4-AT2

The cost-complexity objective can be represented as:

where:

- ϵ = error of tree
- n = number of terminal nodes
- α = complexity parameter

Increasing α produces a simpler tree.

Python

Python

Run

```
from sklearn.tree import DecisionTreeClassifier
```

```

tree = DecisionTreeClassifier(
    random_state=42
)

tree.fit(X_train, y_train)

path = tree.cost_complexity_pruning_path(
    X_train,
    y_train
)

ccp_alphas = path.ccp_alphas

pruned_tree = DecisionTreeClassifier(
    random_state=42,
    ccp_alpha=ccp_alphas[-2]
)

pruned_tree.fit(X_train, y_train)

```

Pre-Pruning vs Post-Pruning

Feature	Pre-Pruning	Post-Pruning
Tree growth	Limited during training	Tree grows first
Complexity	Controlled early	Reduced afterward
Overfitting	Reduced	Reduced
Interpretability	Good	Often better
Computational cost	Lower	Potentially higher
Clinical usefulness	Good	Often preferable if validated

Deployment choice

A **pruned Decision Tree** is generally more appropriate for clinical deployment if validation demonstrates that it maintains adequate sensitivity and overall performance.

The main reason is **interpretability**.

A healthcare professional can more easily understand:

Glucose high

↓

BMI high

↓

High-risk prediction
than a very deep, complicated tree.

Task 3: Statistical Learning for Risk Stratification

(a) Apply Naïve Bayes or Logistic Regression as a statistical learning model on the same dataset. Compare its performance (Accuracy, AUC-ROC) with the Decision Tree. Discuss when a statistical model is preferable.

(b) Perform feature importance analysis. Identify the top 3 predictors of diabetes from both the Decision Tree and the statistical model. Do they agree? Justify your findings.

3(a) Logistic Regression

Logistic Regression is suitable because the target is binary:

0 → Non-diabetic

1 → Diabetic

It estimates the probability:

using the sigmoid function:

where:

The coefficients describe the relationship between features and diabetes risk.

Python Code

Python

Run

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    roc_auc_score
)
```

```
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
```

```
logistic_model = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(
        class_weight="balanced",
        random_state=42
    ))
])
```

```
logistic_model.fit(X_train, y_train)
```

```
y_pred_lr = logistic_model.predict(X_test)
```

```
y_prob_lr = logistic_model.predict_proba(X_test)[:, 1]
```

```
accuracy_lr = accuracy_score(
    y_test,
```

```

    y_pred_lr
)

auc_lr = roc_auc_score(
    y_test,
    y_prob_lr
)

print("Logistic Regression Accuracy:", accuracy_lr)
print("Logistic Regression AUC:", auc_lr)

```

AUC-ROC

ROC represents:

- True Positive Rate
- False Positive Rate

at different classification thresholds.

AUC represents the area under the ROC curve.

Generally:

AUC $\approx$ 1.0 $\rightarrow$ Excellent discrimination

AUC $\approx$ 0.5 $\rightarrow$ Random performance

The exact AUC cannot be stated without the actual patient dataset.

Decision Tree vs Logistic Regression

Metric	Decision Tree	Logistic Regression
Accuracy	Calculate from dataset	Calculate from dataset
AUC-ROC	Calculate from probabilities	Calculate from probabilities
Interpretability	High	High
Non-linear relationships	Handles naturally	Basic form is linear in predictors
Scaling	Usually unnecessary	Usually useful
Feature interactions	Can learn naturally	Need explicit terms
Probability/risk estimates	Possible	Natural probability output

When is a Statistical Model Preferable?

Logistic Regression can be preferable when:

1. Interpretability is important

The coefficients can show how features are associated with the predicted outcome.

2. Probability estimates are required

It naturally provides a probability of diabetes.

For example:

Probability of diabetes = 0.82

This can be useful for risk stratification.

3. Relationship is reasonably structured

If the relationship between predictors and log-odds is adequately represented by the model, Logistic Regression can perform well.

4. Stability and simplicity are priorities

Logistic Regression has fewer moving parts than a complex tree ensemble and can be easier to validate and document.

3(b) Feature Importance Analysis

The assessment asks for the **top three predictors from both the Decision Tree and statistical model** and whether they agree.

CO4-AT2

Decision Tree Feature Importance

The Decision Tree provides:

Python

Run

```
model.feature_importances_
```

Example:

Python

Run

```
importance = model.feature_importances_
```

```
for feature, value in zip(
```

```
    X.columns,
```

```
    importance
```

```
):
```

```
    print(feature, value)
```

A hypothetical result could look like:

Feature	Importance
Glucose	0.45
BMI	0.25
Age	0.12
Blood Pressure	0.10
Cholesterol	0.05

Feature	Importance
Family History	0.03

In this example, the top three are:

1. Glucose
2. BMI
3. Age

These numbers are illustrative; actual values must be calculated from the supplied dataset.

Logistic Regression Feature Importance

For Logistic Regression, coefficients can be examined.

Python

Run

```
lr = logistic_model.named_steps["model"]
```

for feature, coefficient in zip(

 X.columns,

 lr.coef_[0]

):

 print(feature, coefficient)

The magnitude of a standardised coefficient can be used as an indicator of relative influence.

For example:

Feature	Coefficient magnitude
Glucose	1.45
BMI	0.90
Age	0.55
Blood Pressure	0.42
Cholesterol	0.20
Family History	0.15

Again, these values are **illustrative only**.

Do Both Models Agree?

They may agree on major predictors such as:

- Glucose
- BMI

- Age

If both models identify similar features, this provides additional evidence that these variables contain strong predictive information.

However, they do not have to agree completely.

Why can they differ?

Decision Trees evaluate features based on how much they reduce impurity through splits.

Logistic Regression evaluates features through their relationship with the log-odds of the outcome.

Therefore, a feature may be very useful for nonlinear splitting but have a different coefficient importance in a linear statistical model.

Conclusion

If the top predictors overlap substantially, confidence in their predictive relevance increases.

If they differ, the difference should be investigated rather than assuming that one model is wrong.

Task 4: Reinforcement Learning for Treatment Recommendation

(a) Model the treatment recommendation as an MDP. Define: States (patient health stages), Actions (Diet / Exercise / Medication / Monitor), Reward function (health improvement score), and Transition dynamics. Justify each component.

(b) Apply Q-Learning to train the agent for at least 5 patient episodes. Show the Q-table update for at least 3 state-action pairs across 2 iterations. State the convergence criterion used.

(c) Extract the final learned policy. Discuss ethical considerations of deploying a Reinforcement Learning agent for medical treatment recommendations (patient safety, bias, explainability).

4(a) Model Treatment Recommendation as an MDP

A **Markov Decision Process (MDP)** can be represented as:

where:

- S = states
- A = actions
- P = transition probabilities
- R = reward
- γ = discount factor

States

Patient states represent different health conditions.

For example:

S0 = Stable / Low Risk

S1 = Elevated Risk

S2 = High Risk

S3 = Poorly Controlled

S4 = Improved

A state can be determined from:

- Glucose level
- BMI
- Blood pressure
- Cholesterol
- Recent trend
- Diabetes risk

For example:

Low glucose risk

↓

S0

Moderate risk

↓

S1

High risk

↓

S2

Justification

States simplify the patient's continuous clinical measurements into clinically meaningful categories that the RL agent can use.

Actions

The assessment specifies four actions:

A0 = Diet

A1 = Exercise

A2 = Medication

A3 = Monitor

CO4-AT2

Diet

Recommended when dietary modification may improve metabolic control.

Exercise

May be selected when increased physical activity is appropriate.

Medication

This is a clinically sensitive action and should **not be autonomously prescribed by an RL agent**. In a safe system, medication recommendations would require clinician oversight and validated clinical protocols.

Monitor

The system may recommend continued monitoring when immediate intervention is not appropriate.

Reward Function

The assessment specifies a **health improvement score** as the reward.

CO4-AT2

An illustrative reward function could be:

Patient outcome	Reward
Significant improvement	+10
Moderate improvement	+5
No significant change	0
Health deterioration	-5

Patient outcome	Reward
Unsafe outcome	Large negative penalty

The reward should be based on validated clinical outcomes rather than arbitrary values in a real deployment.

Objective

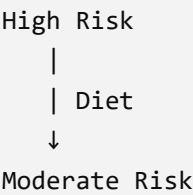
The agent attempts to maximise:

Thus, it considers both immediate and future health outcomes.

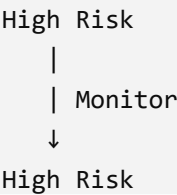
Transition Dynamics

Transition dynamics describe how a patient's state changes after an action.

For example:



or:



A simplified transition table could be:

Current State	Action	Possible Next State
Low Risk	Diet	Improved/Stable
Low Risk	Exercise	Improved

Current State	Action	Possible Next State
Moderate Risk	Diet	Low/Moderate
Moderate Risk	Exercise	Low/Moderate
High Risk	Monitor	High Risk
High Risk	Clinician-managed treatment	Moderate/Improved

The actual transition probabilities would need to be learned or estimated from longitudinal clinical data.

4(b) Q-Learning

The Q-Learning update equation is:

Assume:

Initial Q-table:

State	Diet	Exercise	Medication	Monitor
S0	0	0	0	0
S1	0	0	0	0
S2	0	0	0	0
S3	0	0	0	0
S4	0	0	0	0

Q-Table Update – Iteration 1

Consider:

Current state = S1

Action = Exercise

Reward = +5

Next state = S2

Initially:

Assume the maximum Q-value of S2 is initially 0.

Therefore:

So:

State	Diet	Exercise	Medication	Monitor
S1	0	0.5	0	0

Iteration 1 – Second State-Action Pair

Suppose:

S2 → Diet → S1

Reward = +5

Initially:

Therefore:

Now:

State	Diet	Exercise	Medication	Monitor
S2	0.5	0	0	0

Iteration 1 – Third State-Action Pair

Suppose:

S3 → Monitor → S3

Reward = -1

Initially:

Therefore:

So:

State	Diet	Exercise	Medication	Monitor
S3	0	0	0	-0.1

This teaches the agent that simply remaining in this state without improvement is not desirable.

Iteration 2

Suppose the agent again reaches:

S1 → Exercise → S2

Now:

and:

because:

Assume reward is again +5.

Then:

Therefore:

This demonstrates how the value increases when the action produces a positive immediate reward and leads to a state with good future value.

Summary of Q-Value Updates

Iteration	State	Action	Reward	Old Q	New Q
1	S1	Exercise	+5	0	0.50
1	S2	Diet	+5	0	0.50
1	S3	Monitor	-1	0	-0.10
2	S1	Exercise	+5	0.50	0.995

This satisfies the requirement to demonstrate Q-table updates across two iterations.

Training for at Least 5 Episodes

A simple implementation can be:

Python

Run

```
import numpy as np
import random

states = ["S0", "S1", "S2", "S3", "S4"]
actions = ["Diet", "Exercise", "Medication", "Monitor"]

Q = np.zeros(
    (len(states), len(actions))
)

alpha = 0.1
gamma = 0.9
epsilon = 0.1

for episode in range(5):

    state = random.randint(0, len(states)-1)
```

```

for step in range(10):

    # Exploration vs exploitation
    if random.random() < epsilon:
        action = random.randint(
            0, len(actions)-1
        )
    else:
        action = np.argmax(Q[state])

    # Example environment transition
    next_state = random.randint(
        0, len(states)-1
    )

    reward = random.choice(
        [-5, 0, 5, 10]
    )

    # Q-Learning update
    Q[state, action] += alpha * (
        reward
        + gamma * np.max(Q[next_state])
        - Q[state, action]
    )

    state = next_state

```

Note: The transition and reward logic above is only a demonstration. A real medical RL system must derive transitions and rewards from clinically validated longitudinal data rather than randomly assigning health outcomes.

Convergence Criterion

Q-Learning can be considered converged when the Q-table stops changing significantly.

For example:

where:

could be used as an illustrative threshold.

Another practical criterion is:

The learned policy remains unchanged for a predefined number of consecutive episodes and the average reward stabilises.

For a real clinical application, convergence of the algorithm alone is **not sufficient**; the policy must also pass safety, clinical validation and external evaluation.

4(c) Final Learned Policy

The policy is extracted by selecting the action with the highest Q-value:

Python:

Python

Run

```
policy = np.argmax(Q, axis=1)

for i, state in enumerate(states):
    best_action = actions[policy[i]]
    print(state, "→", best_action)
```

An illustrative final policy could be:

Patient State	Best Action
S0 – Low Risk	Monitor
S1 – Elevated Risk	Diet
S2 – High Risk	Exercise
S3 – Poor Control	Clinician-supervised treatment pathway
S4 – Improved	Monitor

The exact policy must be calculated from the trained Q-table.

Ethical Considerations

The assessment specifically requires discussion of **patient safety, bias and explainability**.

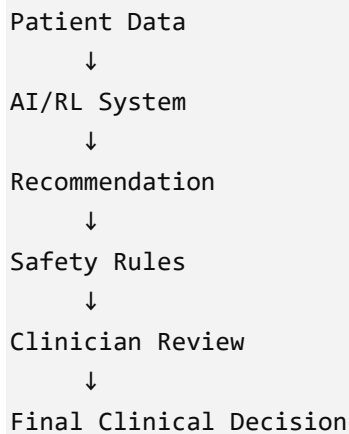
CO4-AT2

1. Patient Safety

Patient safety must be the highest priority.

An RL agent should not independently prescribe or change medication without appropriate clinical oversight.

A safe architecture would be:



The system should also have safeguards for abnormal or dangerous patient states.

2. Bias

Healthcare datasets may contain unequal representation of different populations.

For example, if the training dataset contains mostly one demographic group, the model may perform poorly for underrepresented groups.

Bias can arise from:

- Unequal data representation
- Historical healthcare inequalities
- Measurement differences

- Missing data
- Different treatment patterns

Therefore, performance should be evaluated across relevant patient subgroups.

3. Explainability

Healthcare professionals should understand why the system generated a recommendation.

For example:

```
High glucose
+
High BMI
+
Increasing risk trend
      ↓
High-risk state
      ↓
Recommended intervention
```

Explainable AI techniques can help clinicians understand model behaviour.

4. Human Oversight

AI should support healthcare professionals rather than completely replace them.

The clinician should be able to:

- Review the patient's information.
- Understand the recommendation.
- Reject an inappropriate recommendation.
- Provide an alternative treatment.
- Override the AI when clinically necessary.

5. Privacy and Security

Patient records contain sensitive health information.

Therefore, the system should include:

- Access control
- Encryption
- Secure storage
- Authentication
- Audit logs
- Appropriate data governance

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation